

Local RAG System

NLP Project — Technical Report

FastAPI · ChromaDB · Ollama (Gemma 3) · Docker

2023170599 — مصطفى احمد نبيل ماهر احمد عيد
2023170703 — ياسين هاني ماجد محمد مروان
2023170702 — ياسين مصطفى وهب محمد راشد
2023170751 — يوسف وائل محمد محمود
2022170125 — حبيبة أسامة سيد محمد

1. Executive Summary

A fully containerized RAG system that ingests unstructured documents (PDFs, DOCX), embeds them with sentence-transformers, stores vectors in ChromaDB, and answers natural-language queries by injecting retrieved context into a local LLM (Gemma 3 via Ollama) — all served through a FastAPI backend and deployed with a single docker-compose up command. The use case is CV querying: given a messy, unstructured CV, the system answers recruiter-style questions about the candidate.

2. System Architecture

The pipeline follows a strict MVC separation across dedicated modules. On ingestion, parser.py extracts text page-by-page from PDFs (PyMuPDF) or paragraph-by-paragraph from DOCX files (python-docx), attaching source and page metadata to each chunk. chunker.py then splits the text using LangChain's RecursiveCharacterTextSplitter and embedder.py generates 768-dimensional vectors using BAAI/bge-base-en-v1.5, which are upserted into a ChromaDB persistent collection with cosine similarity.

At query time, routers/query.py receives the request and calls rag_service.py, which embeds the query, retrieves the top-k most similar chunks from ChromaDB via retriever.py, injects them into a prompt, and sends the prompt to Ollama's HTTP API (gemma3 model). The structured response — answer + source chunks + scores — is returned via a Pydantic model.

3. Chunking & Embedding Justification

Documents are chunked with a size of 500 characters and 50-character overlap. The 500-char limit is large enough to hold a complete semantic unit (e.g. a full work experience entry) while staying within embedding model limits. The 50-char overlap prevents boundary losses — a job title and its company name will always co-appear in at least one chunk. The splitter prioritizes paragraph breaks, then sentences, then words, so it respects semantic units rather than applying hard cuts.

The embedding model was upgraded from all-MiniLM-L6-v2 (384-dim) to BAAI/bge-base-en-v1.5 (768-dim) after Phase 4 evaluation revealed the smaller model failed to bridge the vocabulary gap between natural-language queries and CV text. BGE-base is specifically optimized for retrieval tasks and runs on CPU without a GPU.

4. API Documentation

POST /api/query

Accepts a natural-language query and returns an LLM-generated answer with retrieved source chunks and similarity scores.

Request	Response
<pre>{ "query": "Where does the candidate work?", "top_k": 3 }</pre>	<pre>{ "query": "...", "answer": "Instabug, Cairo", "chunks": [{ "text": "...", "source": "cv.pdf", "page": 1, "score": 0.51 }] }</pre>

GET /api/health

Returns { "status": "ok" } — used to verify the container is running.

5. Docker Deployment

```
git clone <repo-url> && cd rag-project
docker-compose up --build
# API at http://localhost:8000/docs
```

Two services start: rag-api (FastAPI + ChromaDB + embedder) on port 8000, and ollama (Gemma 3 inference) on port 11434. The chroma_db/ volume is pre-ingested and mounted — no manual ingestion step is needed. First run downloads Gemma 3 (~3.3 GB); subsequent runs are instant.

6. Evaluation & Error Analysis

The system was tested with targeted edge-case queries against the ingested CV. Three distinct failure modes were identified:

- **Numerical aggregation failure:** "How many years of experience does the candidate have?"
the retriever matched the freelance chunk (2019–2020) instead of the summary paragraph stating "4+ years", causing the LLM to hallucinate "1+ years". Root cause: weak semantic overlap between the query and the summary. Fix: hybrid BM25 + vector retrieval.
- **Ordinal reasoning failure:** "What projects did the candidate work on at their second job?"
the system returned Instabug (first job) instead of Synapse Analytics (second). RAG has no concept of chronological ordering; embedding scores rank by relevance, not position. Fix: tag chunks with job-order metadata during ingestion.
- **Semantic negation failure:** "What programming languages does the candidate NOT know?"
the correct skills chunk was retrieved, but the LLM misapplied the negation constraint and included Bash (a known skill) in the answer. Embedding models ignore negation entirely. Fix: chain-of-thought prompting to reason about negation before answering.